(THIS WAS THE MACHINE TRANSLATION)

c R A R k
(First & Fastest RAR Cracker)
v. 5.5

1. PURPOSES AND CHARACTERISTICS

The cRARk program is designed to determine a forgotten password for RAR-archives. This program operates adequately with RAR-archives versions 2.x, 3.x, 4.x, 5.x.

To proceed with cRARk program you need a computer with the Pentium compatible processor with SSE2 support or later. It is recommended to use as powerful processor as possible (the code is optimized for Intel Core 2/Core i7, AMD Athlon/Bulldozer). From v. 3.2, cRARk supports NVIDIA GPU with CUDA support and computing capability 1.1. From v. 3.4, cRARK also supports OpenCL both on NVIDIA and ATI/AMD Radeon GPUs.

cRARk is the tool for professionals, no GUI or great service is provided. But it tries to maximize your abilities for passwords definition and to minimize search time. cRARk uses Password Definition Library (PDL), a very powerful tool allowing you to define rules to generate passwords.

The rate of password search should be about of 500 password per second on modern powerful computer, so finding the 6-characters password of lower case Latin letters will need about a month. In the GPU the rate could achive 100000 p/s. Rate for dictionary attack is about 200 pass/sec (GPU is not supported).

2. REQUIREMENTS FOR THE INPUT ARCHIVE

The following requirements to testing RAR archive are to be met:

- There is at least one encrypted file.
- The password should be not longer than 28 symbols (RAR 3.x-4.x).

In case of solid-archives, the first file should satisfy these requirements. Therefore, if the files in archive were encrypted with the different passwords, the password for first file will be found.

cRARk must be working with selfextracting (sfx), multivolume archives and archives with encrypted headers.

From version 3.2, cRARk supports passwords computations on GPU cards (CUDA and OpenCL technology). Current limitations are:

- Only NVIDIA cards with computing capability 3.0 and up is supported. AMD/ATI GPUs are supported from 5xxx/6xxx serie.
- You need the latest drivers with CUDA and/or OpenCL support
- GPU engine is activated if password definition contains '*' (repetition symbol). All another passwords are tested on CPU.
- Files with advanced compression options (-mc) not supported
- RAR 2.x archives also not supported

3. Working with the program

To run the program YOU NEED TO CREATE PASSWORD DEFINITION FILE firstly (see section 4).

This is a command-line utility! You should run it from command (DOS) prompt.

To run the program you should use:

```
CRARK [options] archive
```

The found password is printed in such a form:

```
truepass - CRC OK
```

Next it is repeated in hexadecimal PDL-like form (see section 4.2.1).

All other messages ARE NOT passwords and are intended as progress

indication of the program.

Options in this mode are:

-lXX	set password length to XX at least (XX = 0..255, XX = 1 by default). This parameter affects password length only when '*' is used in its definition (see section 4.2.1);
-gXX	set password length to XX at most (XX = 0..255, XX = 8 by default)
-pXXXX	set the name of password definition file ("PASSWORD.DEF" by default).
-b	perform benchmarking
-v	debug mode (see section 5). It may be used to show character sets in use. This option generates also all the passwords according to with their definition; it does not test but prints them, so you can check their validity.
-c	disable GPU support (use CPU only)
-dXX	set GPU delay (XX = 0..5), 0 - fastest, 5 - slower, but smoothest
-nXX	set GPU device number (0 by default)
-fXYZ	(advanced) set crypto functions #X,Y,Z. Some information about available functions is printed by -b option.

4. USING OF PASSWORD DEFINITION FILE

Password definition file is the main control file. Its translation and processing are the main task of PDL library. Its format doesn't depend on application, to which PDL is linked, so this library can be used for any password searching program.

Password definition language (PDL) is designed for flexible and powerful control of password generation. It allows password definition creation using all known information about a password which substantially reduce the time of password searching. The typical example of such information is : "I remember that my password consist of two words separated by the one of the signs "-", "_", "=", "," and the first letters of the words are in the uppercase" (see example 1 to see how it looks in PDL).

The main idea of PDL language is component-based description of password. The above example has three components - first word, separator (symbol) and the second word. In most cases is it possible to divide the password verbal description into components.

The syntax of PDL language is close to the regular expressions syntax. Your knowledge of regular expressions will help you understand PDL language easily.

There are three main components type in PDL language - charsets, words and generators.

4.1. Password definition file format

Password definition file is just text file in UTF-16 or UTF-8 format (Windows) or UTF-8 (Linux, Mac OS) and consists of two parts: firstly, dictionary and character set definition, and secondly, passwords definition.

The parts are separated by a line of two '##' symbols:

```
[ <dictionary and character set definition> ]
##
<passwords definition>
```

The first part could be missing, in that case password definition file starts with '##' symbols.

In all other cases the symbol '#' is considered as a start of comment.

Space characters and tabs are ignored in password definition file and may separate any components.

To make it easy, let's look upon the password definition mechanism at first and then character set definition, contrary to their position in password definition file.

4.2. Password definition

This is the main part of the file. IT NECESSARILY EXSISTS IN ANY PASSWORD DEFINITION FILE (PASSWORD.DEF) AFTER THE LINE '##' and presets password generation rules to be checked out later on. It consists of the text lines, each of them gives its own password set and mode of operation, i.e. an algorithm of password search. Each line is independent and is processed separately, herewith the total number of checked passwords is counted up.

Character sets, dictionary words and generators form password definition. They preset one or more characters, which will hold the appropriate position in a password.

4.2.1. Character sets

Character set (charset) is a range of characters, any of them can occupy current position in a password. The supported charsets are:

1) Simple single characters (**a**, **b**, etc.). It means this particular character occupies given position in a password;

2) Shielded characters. If any of special characters can ever occur in the password, it must be shielded with '\'. The meaning is identical with item 1 mentioned above. Among these characters are:

\$, **\.**, *****, **\?**, **\=**	'$', '.', '*', '?', '='
\], **\[**, **\{**, **\}**, **\(**, **\)**	corresponding brackets;
**** (space character)	space character
\xxxx	any Unicode character in hex (X is a hexadecimal digit), like \D9E9
\0	no character. It is usually used in conjunction with "real" character (please find examples below).

Generally, any character can be shielded except hexadecimal digits.

3) Macros of character set. It means that current position in the password can be occupied by any character from the set. These sets are specified in the first part of password definition file (see section 4.3.2) and are denoted as:

$a	lower-case Latin letters (26 letters, unless otherwise specified)
$A	upper-case Latin letters (26 letters, unless otherwise specified)
$!	special characters (32 characters, unless otherwise specified)
$1	digits (10 digits, unless otherwise specified)
$i	lower-case letters of national alphabet
$I	upper-case letters of national alphabet
$o	other user-specified characters
?	any character (i.e. all the characters, included into the macros mentioned above)

NOTE: macros **$v** and **$p** (see section 4.3.4) cannot be used for password definition.

4) Any combinations of the characters mentioned above. It must be written in square brackets. The meaning is identical with item 3 mentioned above. For example:

[$a $A]	any Latin letter
[abc]	a, or b, or c
[$1 abcdef]	hexadecimal digit
[s \0]	s or nothing
[$a $A $1 $! $i $I $o]	this is equivalent to ?

5) Regular repetition character ***.It means the previous character set has to be repeated in related item of the password 0 or more times. For example:

$a *	a password of any length, consisting of lower-case Latin letters
[ab] * c	c, ac, bc, aac, abc, bac, bbc, aaac, ...
[$a $A] [$a $A $1] *	"identifier", i.e. a sequence of letters and digits with a letter at first position

Note that password of zero length (null password) is physically meaningful and is not always the same as no password at all.

The length of repetition is computed in two ways:

- automatically on the base of the defined maximum and minimum password length options. (options. (Note, when there is no '*' in password definition, these parameters are ignored.)
- set explicitly in brackets in such way: *** (length)** or *** (min length, max length)**. Examples:

a b *(0,3) c	will generate 4 passwords: ac, abc, abbc, abbbc
a [bc] *(2) d	abbd, abcd, acbd, accd
$1 *(2,4)	all two-, three- and four-digit numbers

It is recommended to use '*' as wide as possible. This is because it allows to perform the most powerful search. Although the constructions '? *' and '? ? *' seem to be alike from the logic standpoint, the first one will be searched through faster.

Current limitation: '*' can be used only once.

4.2.2. Dictionary words and their modifiers

Contrary to the character set, the words present several consecutive passwords characters. Two dictionaries are supported by PDL library: main (with ordinary words mainly) and user's (where special information can be stored, for example, proper names, dates, etc.), though there is no difference between them.

Dictionary is a text file in both UTF-16 or UTF-8 encoding (Windows) or only UTF-8 (Linux, Mac), consisting of the words, separated by the end-of-line characters. Both DOS-format (CR/LF) and UNIX-format (LF) files can be used. Preferably to use words of the same (lower) case in dictionaries (to increase search rate, among other factors). To sort out words by its length is also desirable.

Thus, there are two macros:

$w	a word from the main dictionary
$u	a word from the user dictionary

As is known altered words are often used as passwords. So a whole set of word modifiers is introduced to determine such passwords. Among these are:

.u (upper)	to upper-case
.l (lower)	to lower-case
.t (truncate)	to truncate up to the given length
.c (convert)	to convert the word
.j (joke)	to upper-case some letters
.r (reverse)	to reverse the word
.s(shrink)	to shrink the word
.p (duplicate)	to duplicate the word

Modifiers may have parameters, written in round brackets. For modifiers, meant for single letters use, it is possible to preset a number of the letter as a parameter. Lack of parameters or null parameter means "the whole word".Then, letters can be numerated from both the beginning of the word and the end of the word. The end of the word is denoted with the character '-' .

There are only three such modifiers for today: **.u**, **.l**, **.t**. So, use:

.u or **.u(0)**	to upper-case the whole word (PASSWORD)
.u(1), **.u(2)**	to upper-case only the first (the second) letter (Password, pAssword)
.u(-), **.u(-1)**	to upper-case the last (the next to last) letter (passworD, passwoRd)
.t(-1)	to truncate the last letter in the word (passwor)

The other modifiers operate with the whole words only and their parameters give the way of modification. The following modifier parameters are specified for today:

.j(0) or **.j**	to upper-case odd letters (PaSsWoRd)
.j(1)	to upper-case even letters (pAsSwOrD)

.j(2)	to upper-case vowels (pAsswOrd)
.j(3)	to upper-case consonants (PaSSWoRD)
.r(0) or **.r**	to reverse the word (drowssap)
.s(0) or **.s**	to reduce the word by discarding vowels unless the first one is a vowel (password -> psswrd, offset -> offst)
.p(0) or **.d**	to duplicate the word (passwordpassword)
.p(1)	to add reversed word (passworddrowssap)
.c(<number>)	to convert all the letters in the word according to the appropriate conversion string (see section 4.3.3)

All the modifiers operate adequately with both Latin and national letters, provided that the rules of national character sets definition are observed. Clearly there can be more than one modifier (the number of consecutive modifiers is limited by 63, which is unlikely to be exceeded). For example: (let $w mean a "password"):

$w.u(1).u(-)	PasswordD
$w.s.t(4)	pssw
$w.t(4).s	pss

4.2.3. Generators

Generator is the last possible component type and it generates several passwords of different length from the given symbols in current position. Generators are denoted by **{** and **}** symbols followed by generator type, like **{abc}.u**. The opening bracket indicates the position of beginning the generator, and the closing one - the ending position.

1) The default generator - permutation brackets **{ }**

This generator has no explicit type, i.e. it simply denoted by **{** and **}**

The problem is widely met, when you remember your password, but it is not valid for some reason. Probably, you mistype it. The PDL engine has its own algorithm to restore such passwords. The following typing mistakes are considered: two neighboring letters are swapped (psasword), a letter is omitted (pasword), an needless letter is inserted (passweord) or one letter is replaced with another (passwird). Such password changes will be referred to as permutations.

To indicate the beginning and the end of the password section where permutations could appear, permutation brackets '{' and '}' are used. The bracket '}' can be followed by a number of permutations (1 by default), separated by a dot. The physical meaning of the number of permutations is the number of simultaneously introduced mistakes. For example:

{abc} - 182 (different) passwords will be obtained, including:

bac, acb	2 swaps
bc, ac, bc	3 omissions
aabc, babc ...	4 * 26 - 3 insertions
bbc, cbc ...	3 * 25 replacements

abc	the word itself

{password}.2 - the following words will be generated: psswrod, passwdro, paasswor, etc.;

{$w} - all the words, containing one mistake, from the main dictionary.

Notes:

a) It is obvious that some passwords will be obtained more than once, so the larger is the number of permutations, the larger is the number of replicas. Efforts were made in this program to reduce replicas, but they are purely empirical and were made for two permutations at most. In other words, for the large numbers there is no certainty that a particular password cannot be discarded erroneously.

b) For insertion and replacement one should know the set of characters to be inserted or replaced. In the event this set is not specified explicitly (see section 4.3.4), this program forms it automatically for character sets, in relation to standard set these characters are from (i.e. for **{password}** $a will be inserted, for **{Password}** [$a $A] will be inserted). The similar operation with words is performed, based on the first word from the dictionary with modifiers being taken into account. In the event this set is specified explicitly, it is just the set to be used.

2) Upper and lowercase generators **{}.u, {}.l**

These generators will generate all possible upper or lower-case combinations from the given symbols, like:

{abc}.u	abc, Abc, aBc, abC, ABc, AbC, aBC, ABC
{AbC}.l	AbC, abC, Abc, abc
{$w}.u	All main dictionary words with all possible upper-case conversion

3) Conversion generator **{}.c**

Like the above generator, this one is used to generate all possible combination of word using the appropriate convert table (see section 4.3.3) written in round brackets.

Let .c(0) convert table defines conversion of letters to similar symbols, then

{password}.c(0)	password, pa$sword, pas$word, passw0rd, pa$$word, pa$sw0rd, pas$w0rd, pa$$w0rd

Of course, there may be several generators in the password description. So, the definition like

{my{good}password}.u

may occur.

4.3. Dictionaries and character sets definitions

All the definitions are set in the beginning of password definition file up to the characters '##'.

4.3.1. Dictionaries definition

The main and user dictionaries in use (see section 4.2.2) are initially

defined as usual. It is necessary only if you are going to use words from the dictionaries when defining passwords, i.e. $w or $u.

The dictionaries are given as follows:

$w = "main.dic"	# main dictionary
$u = "c:\\dict\\user.dic"	# user dictionary

File name is to be quoted, the path characters are to be shielded.

4.3.2. Definition of the character sets in use

The character sets in use are defined as usual. They are classified in two groups: predefined and user-defined.

Predefined sets include:

$a	lower-cased Latin letters, 26 letters in all
$A	upper-cased Latin letters, 26 letters in all
$!	special characters (32 characters in all) {}:"<>?[];\',./~!@#$%^&*()_+`-=|
$1	digits, 10 digits in all

User-defined sets include:

$i	lower-cased letters of national alphabet
$I	upper-cased letters of national alphabet
$o	additional character set (for example, any non-typable characters)

Character sets are defined as follows:

$<charset> = [<single characters or character sets>]

In other words, character set is written as combination of characters (see section 4.2.1), for example:

```
$i = [$a   ä ö ü ß ]
$o = [$! $1 \20ac]
```

NOTES:

1) Any character sets are allowed to be defined, including pre-defined. For example, you may include additional characters, such as euro sign € (\20ac) into the set $!

2) When the sets $i and $I are being defined, the function of switching between lower/upper case is defined automatically. So it is important to have letters being ordered uniformly in these sets.

The full character set '?', consisting of [$a $A $1 $!$i $I $o] (just such an order is of importance in the next section), is never formed until all the characters are defined.

4.3.3. Definition of conversion modifiers

The conversion modifiers **.c** may be defined (see section 4.2.2) are described the conversion rule of one character to another. It is performed with the line of the form: **.c(<number>) = "<conversion string>"**

The conversion string is written in such way **"o=0|l=1|s=5|b=6"**,

where the conversion pairs are divided with '|' symbol and at left side of '=' is written a character to be converted to character on the right side of '='.

For example, let the $w is "lower", then $w.c(0) = "10wer". The characters '\' and '|' are to be shielded in conversion string. The numbers of modifiers may vary from 0 to 255.

4.3.4. Special character sets definition

Among special character sets are:

$v	a set of vowels (in all alphabets being used). It is needed only when .s and .j modifiers are used
$p	a set for insertion and replacement for permutation brackets. It is needed only if automatic generation of this set does not suit you for some reason (see section 4.2.3)

These sets are defined in a similar way to the other character sets.

4.3.5.Minimal and maximal password length

The minimal and maximal password length settings affect only password definition with regular repetition symbol and not affected other password definition including simply charsets, words or generators. To set the default password range, use keywords **min** and **max**, like:

min = 1	generate passwords from one character long
max = 5	generate passwords up to 5 characters long

4.4. USEFUL EXAMPLES OF PASSWORD DEFINITION

1) I remember that my password consist of two words separated by the one of the signs "-", "_", "=", "," and the first letter of the words are in the uppercase

```
$w = "dictionary.txt"
##
$w.u(1) [\-_\=,] $w.u(1)    # "-", "=" needs to be shielded
```

It should be mentioned that both $w are distinct words, so a total of 20000 * 4 * 20000 = 1 600 000 000 passwords (if there are 20000 dictionary words) will be generated.

If you're not sure if first letter be in uppercase or not, you need to make all possible combination writing four-line definition:

```
$w = "dictionary.txt"
##
$w.u(1) [\-_\=,] $w.u(1)
$w.u(1) [\-_\=,] $w
$w [\-_\=,] $w.u(1)
$w [\-_\=,] $w
```

Additionaly, if you may insert from 1 to 3 symbols between the words, the definition is changed as:

```
$w = "dictionary.txt"
##
$w.u(1) [\-_\=,] *(1,3) $w.u(1)
```

2) Most common definition - brute-force search using only lower-case Latin letters.

```
$a *
```

You also can use **min=** and **max=** settings or -l and -g options to define password length.

3) The same as the above, but using both cases Latin letter

```
[$a $A] *
```

4) Simplest dictionary attack

```
$w = "dictionary.txt"
##
$w
```

5) Dictionary attack with all possible upper/lower-case combinations:

```
$w = "dictionary.txt"
##
{$w}.u
```

5) Let me cite ZEXPL2L program specification: "Let you have an archive with the password looking like "Heaven!!!", but you have forgotten, how many !s were there in the end and what kind of letters lower- or upper-cased were used: "HeAvEn!!!", "Heaven!" or "HeAven!!!!". But fortunately you remember your password to be 10 characters at most and 7 characters at least." This password will be written in PDL language as follows:

```
min = 7
max = 10
##
{heaven}.u ! *
```

Instead of min= and max= -l and -g option can be used.

Suppose that among other things you have mistaken while typing the main part of the password. So the following one is worth attention:

```
min = 7
max = 10
##
{{heaven}}.u ! *
```

6) One more citation from the same specification: "Let you have two variants of the password string: "myprog", "MyProg", "my_prog" and "My_Prog". It will be written as:

```
[mM] y [_ \0] [pP] rog
```

7) Password consists of exactly six letters from national alphabet:

```
$i $i $i $i $i $i
```

But

```
$i *(6)
```

are far more efficient.

8) The date in DDMMYYYY format:

```
$1 * (8)
```

But

```
##
[0123] $1 [01] $1 [12][90] $1 *(2)
```

will generate much less passwords.

9) You remember your password to be "MyVeryLongGoodPassword", but it

is not valid for some reason. Try to use the following ones:

{MyVeryLongGoodPassword}	2360 passwords in total
{MyVeryLongGoodPassword}.2	2 785 406 passwords

10) You know you password to be a meaningful word with a digit inserted elsewhere. The definition file is:

```
  $p = [$1] # the insertion set is defined as a set of digits
##
{$w}
```

11) Syllable attack. You need to set up a dictionary of possible syllables of your language and then to search through all the meaningful words by proceeding as follows:

```
  $u # monosyllabic words
$u$u # disyllabic words
$u$u$u # etc.
$u$u$u$u
```

12) In order to run your program in parallel on two computers, give them the following definition files:

[abcdefghijklm] $a *	for the first one
[nopqrstuvwxyz] $a *	for the second one

Proceed similarly with n computers.

5. Possible problems (FAQ)

1. How to break and then to continue the search.

The program may be broken painlessly once the message "Testing XX-chars passwords..." is displayed, and then the search may be continued with - lXX option (both XX are equal).

2. How to resume search from the password XXX?

Sorry, no way. I's implemented in Parallel Password Recovery.

3. The program has been searching for 10 days, but my password is not yet at hand.

Alas! It can't be helped. May be your password is too long, or the search set is wrong. Additional information on the password is necessary.

4. There are files with different passwords in the archive. What am I to do?

Just remove (using RAR) files with already known passwords.

5. I have tested your program. To my mind, your program is nothing but utter error, it cannot even find "aaa2"-like password.

RTFM. Distributive file password.def searches through only lower-cased Latin letters. Change your password definition to "[$a $1] *" and everything will be ok.

6. I've got beginning of one file from archive in plain text. Will it be useful to me?

No. At least, I couldn't use it. Could you? RAR encryption sources are available in WinRAR distribution.

7. I'd like to optimize your program. How can I get the sources?

You don't need them. Take UnRar sources and optimize the SetCryptKeys() function. Next contact me.

8. Is there any option to save program operation log?

Probably, you have never dealt with UNIX. Use:

```
crark [options] > file
```

If you don't like this, use "tee" utility.

9. Is it possible to speed-up dictionary attack?

Yes, just sort your dictionary by the words size

10. Your distribution kit is packed with a password in itself!!! I do not find it funny!

You are reading this file, so you have solved this problem.

11. I need GUI, multicore support, pause/resume etc.

cRARk is the free program, and I have no time to support such features. To find all you need, please look at Parallel Password Recovery which licensed cRARk and PDL engine.

12. How to use my GPU card to recover passwords?

You need:
a) NVIDIA GPU from GeForce 6xx serie or AMD GPU from 6xxx serie (GCN is preferred)
b) latest drivers
c) the password definition should contain '*' symbol
d) only RAR 3.x-5.x archives are supported, except of files with advanced comperssion (-mc) methods

13. I've got some errors when using my GPU card.

a) Please install the latest drivers!
b) Don't overclock neither GPU nor CPU!
c) On Windows, if the message about the stopped driver appears, please use the file driver-timeout.reg from the distributon archive.

6. On PDL library

PDL 2.0 library is distributed by the author as FREEWARE in the form of source text. The reference to PDL as an obligatory requirement for your programs.

PDL 3.0-4.0 with plenty of new features is not a FREEWARE.

7. How to contact the author

Only by e-mail.

Please don't ask me about how to run the program etc - I have no time to explain individually.

e-mail: pavel@semjanov.com
WWW: http://www.semjanov.com

Program URL is: http://www.crark.net

A lot of free, benchmarked password crackers you'll find at:

http://www.password-crackers.com

cRARk is a FREE program, so all the claims will be rejected. Anyway, I'll be very grateful for pointing out the serious errors, such as:

- the program hangs up while searching (the lack of displayed messages is not an evidence of hangup);
- the program cannot find such password in such archive, although the set of characters in search is specified correctly

I'll be also glad to any constructive suggestions on improvements of program operation.

8. Special thanks and copyright notices

To Eugene Roshal for good encryption algorithm.
To Eugene D. Shelwien for his outstanding ideas about RAR 2.0 cracking.
To John Vandermeersch <vanderme@tornado.be>, Roman Paul and Natalia Leonova for correcting this docs.
To Phil Frisbie, Jr. (pfrisbie@geocities.com) for CPU identification function.
To Andy Polyakov and other guys from OpenSSL for some optimization ideas.

Good luck!

Pavel Semjanov, St.-Petersburg.